Z Á K L A D N Í N U M E R I C K É M E T O D Y

S T A R T R A K E T Y

třetí semestr

Rudolf Žižka
547854

Obsah

1 Představení problému **3**
- 1.1 Cíle řešení problému . . . 3
- 1.2 Způsob řešení . . . 3

2 Teoretický úvod **4**
- 2.1 Použité numerické metody . . . 4
  - 2.1.1 LSODA . . . 4
  - 2.1.2 Runge-Kutta 4 . . . 4
  - 2.1.3 BDF . . . 4
- 2.2 Odvození Heunovy metody . . . 4
  - 2.2.1 Tvar Heunovy metody . . . 4
  - 2.2.2 Následující krok Taylorovým rozvojem . . . 5
  - 2.2.3 Určení parametrů . . . 5
  - 2.2.4 Zpětné dosazení a interpretace . . . 6
- 2.3 Pohybová rovnice . . . 6
  - 2.3.1 Hmotnost rakety v čase . . . 6
  - 2.3.2 Dosazení hmotnosti do pohybové rovnice . . . 7

3 Řešení problému **8**
- 3.1 Analytické řešení . . . 8
  - 3.1.1 Finální rychlost rakety . . . 8
- 3.2 Numerické řešení . . . 9
- 3.3 Určení relativní chyby Heunovy metody . . . 10
- 3.4 Určení relativní chyby metod a jejich porovnání . . . 12

4 Závěr **13**

1 Představení problému

Uvažme jednorozměrný případ startu rakety v homogenním gravitačním poli $g = -9,81\,\mathrm{N\,kg^{-1}}$, směr gravitačního zrychlení je orientován proti směru osy systému $\vec{e}_h$. Při řešení zanedbáme odpor prostředí a změnu gravitačního zrychlení, ke kterým by docházelo při reálném startu. Tah raketového motoru a spotřebu paliva také budeme považovat za konstantní.

Raketa je na začátku startu v klidu a je v počátku souřadnic, $h(0) = 0\,\mathrm{m}$ a $\dot{h}(0) = 0\,\mathrm{m\,s^{-1}}$, má počáteční hmotnost $m_i = 12800\,\mathrm{kg}$ a spaluje palivo rychlostí $r = 130\,\mathrm{kg\,s^{-1}}$ až na konečnou hmotnost $m_f = 4000\,\mathrm{kg}$. Raketový motor generuje konstantní tah o velikosti $F = 250\,\mathrm{kN}$.

1.1 Cíle řešení problému

Úkolem je najít závislost $h(t)$ rakety pro počáteční podmínky analyticky a poté i numericky a získaná řešení porovnat. Pomocí analytického řešení také ověřit Ciolkovského rovnici 1-1 pro finální rychlost rakety za předpokladu $g = 0\,\mathrm{N\,kg^{-2}}$

$$\dot{h}_f = v_e \ln\left(\frac{m_i}{m_f}\right) \tag{1-1}$$

1.2 Způsob řešení

Nejdříve jsme analyticky problém vyřešili pro následné porovnání s numerickým řešením. Poté jsme pomocí knihoven v pythonu určili numerické řešení metodami DBF[1], Runge-Kutta 4 a LSODA[2]. Tyto metody popíšeme jen velmi zhruba jelikož je používáme pouze pro srovnání s naší metodou. V pythonu jsme vytvořili vlastní verzi Heunovy metody a její řešení jsme srovnali s analytickým řešením a řešením podle metod z pythonovských knihoven. Poměřením s analytickým řešením jsme určili relativní odchylku všech metod od reálné hodnoty pro počet kroků metody a u Heunovi metody i v závislosti na koeficientu γ, který bude popsán dále.

[1] Používá Eulerovu metodu.

[2] Používá Eulerovu metodu nebo metodu typu Adams s adaptivním rozdělením časového, evaluačního, intervalu

2 Teoretický úvod

2.1 Použité numerické metody

2.1.1 LSODA

LSODA (Livermore Solver for Ordinary Differential equations with Automatic method switching) je metoda numerického řešení lineárních diferenciálních rovnic prvního řádu, která automaticky přepíná mezi Adamsovým algoritmem a BDF metodou v závislosti na tuhosti řešené soustavy rovnic. Pro tuhé problémy používá Adamsův algoritmus. V opačném případě používá metodu založenou na BDF, která je vhodnější pro tuhé problémy.

Tuhá soustava rovnic se vyznačuje rychlými změnami v čase, pokud tedy u některých metod nezvolíme malý krok, je výsledek velmi náchylný k velké chybě. Námi analyzovaný problém není na řešeném intervalu tuhý.

2.1.2 Runge-Kutta 4

Runge-Kutta 4 (RK4) je metoda čtvrtého řádu pro řešení lineárních diferenciálních rovnic prvního řádu. Každý krok výpočtu zahrnuje čtyři mezivýpočty, proto metoda čtvrtého řádu, a zohledňuje hodnoty derivací v různých bodech intervalu, čímž minimalizuje chybu. Metoda RK4 je jednoduchá a relativně přesná, i když není vhodná pro velmi tuhé problémy, protože může vyžadovat velmi malý krok pro stabilní výpočet.

2.1.3 BDF

BDF (Backward Differentiation Formula) je metoda vhodná pro řešení tuhých soustav lineárních diferenciálních rovnic prvního řádu. Používá hodnoty funkce z předchozích kroků k odhadu nového bodu, což zajišťuje lepší stabilitu pro tuhé problémy. První řád, BDF1, je přímo Eulerova metoda, ta používá jenom první aproximaci kroku a není proto pro tuhé systémy dobrá oproti následujícím řádům metody BDF.

2.2 Odvození Heunovy metody

Heunova metoda používá dvě aproximace pro vypočítání dalšího bodu funkce s krokem h. Musíme tedy počáteční předpis Heunovy metody porovnávat s Taylorovým rozvojem druhého řádu vektorové funkce systému.

2.2.1 Tvar Heunovy metody

Uvažme krok numerické metody $h = t_{n+1} - t_n$ a vektorovou funkci ve tvaru $\vec{f}(\vec{x}, t)$. První aproximace má tvar

$$\vec{k_1} = h\vec{f}(\vec{x}_n, t_n)$$

Druhou aproximaci budeme hledat ve tvaru

$$\vec{k}_2 = h\vec{f}(\vec{x}_n + \alpha\vec{k}_1, t_n + \beta h)$$

která v první aproximaci Taylorovým rozvojem má tvar

$$\vec{k}_2 = h[\vec{f}(\vec{x}_n, t_n) + \alpha\vec{k}_1\vec{f}_{\vec{x}}(\vec{x}_n, t_n) + \beta h\vec{f}_t(\vec{x}_n, t_n) + \vec{\sigma}(h^2)]$$

kde dolní index vyjadřuje parciální derivaci podle dané proměnné a $\vec{\sigma}(h^2)$ je chyba řádu h^2. Jelikož se nám všude objevuje funkce $\vec{f}$ vyčíslená v bodě $(\vec{x}_n, t_n)$ budeme tento parametr pro kratší zápis vynechávat. Následující hodnotu kterou metoda vrátí poté vyjádříme jako lineární kombinaci dvou uvedených kroků ve tvaru

$$\vec{x}_{n+1} = \vec{x}_n + \gamma_1\vec{k}_1 + \gamma_2\vec{k}_2 \tag{2-1}$$

po dosazení odvozených kroků dostáváme pro $\vec{x}_{n+1}$

$$\vec{x}_{n+1} = \vec{x}_n + \gamma_1 h\vec{f} + \gamma_2 h(\vec{f} + \alpha h\vec{f}\vec{f}_x + \beta h\vec{f}_t) + \vec{\sigma}(h^3)$$

A po vytknutí dostáváme finální vztah, se kterým budeme dále pracovat.

$$\vec{x}_{n+1} = \vec{x_n} + (\gamma_1 + \gamma_2)h\vec{f} + \gamma_2 h^2(\alpha\vec{f}_x\vec{f} + \beta\vec{f}_t) + \vec{\sigma}(h^3) \tag{2-2}$$

2.2.2 Následující krok Taylorovým rozvojem

Abychom mohli určit parametry γ_1, γ_2, α a β porovnáme vztah 2-2 s Taylorovým rozvojem druhého řádu vektorové funkce $\vec{f}$

$$\vec{x}_{n+1} = \vec{x}_n + h\vec{f} + \frac{1}{2}h^2(\vec{f}_x\vec{f} + \vec{f}_t) + \vec{\sigma}(h^3) \tag{2-3}$$

2.2.3 Určení parametrů

Porovnáním 2-2 a 2-3 získáme vztahy pro parametry Heunovy metody.

$$\gamma_1 + \gamma_2 = 1$$

$$\gamma_2\alpha = \frac{1}{2}$$

$$\gamma_2\beta = \frac{1}{2}$$

Řešením této soustavy dostaneme tvar parametrů v závislosti na γ_1, tento parametr byl vybrán vzhledem k interpretaci.

$$\alpha = \beta = \frac{1}{2(1-\gamma_1)} \tag{2-4}$$

$$\gamma_2 = (1 - \gamma_1) \tag{2-5}$$

2.2.4 Zpětné dosazení a interpretace

Když parametry dosadíme zpět do vztahu 2-1 dostaneme pro $\gamma_1 \in [0, 1)$

$$\vec{x}_{n+1} = \vec{x}_n + \gamma_1 h\vec{f} + (1-\gamma_1)h\vec{f}\left(\vec{x}_n + \frac{1}{2(1-\gamma_1)}h\vec{f}, t_n + \frac{1}{2(1-\gamma_1)}h\right) \quad (2\text{-}6)$$

Další krok pro γ_1 blízké 1 se téměř rovná Eulerově metodě a pro hodnoty blízké 0 úplně zanedbává první aproximaci a používá pouze druhou aproximaci. Když se budeme přibližovat k Eulerově metodě velmi vzroste velikost koeficientů α a β, můžeme tedy předpokládat, že přesnost metody se zhorší.

Pro hodnotu $\gamma_1 = 1$ metoda přejde na Eulerovu metodu, nebudeme ji ale moct implementovat, protože bychom při výpočtu α a β dělili nulou. Museli bychom tedy vytvořit speciální kód pro Eulerovu metodu.

2.3 Pohybová rovnice

Pohyb rakety popisujeme pouze pomocí jedné souřadnice $h(t)$, pohybová rovnice je tedy čistě skalární. Při jejím sestavení budeme vycházet z druhého Newtonova zákona

$$m(t)\ddot{h}(t) = \sum_i F_i$$

kde $m(t)$ je proměnná hmotnost rakety, $\ddot{h}(t)$ je druhá časová derivace $h(t)$[3], a F_i jsou síly působící na raketu.

Na raketu podle zadání působí gravitační síla, kterou můžeme napsat ve tvaru

$$F_1 = -m(t)g$$

Raketový motor v souladu s třetím Newtonovým zákonem nepřímo působí na raketu silou

$$F_2 = F$$

Zpětným dosazením dostáváme diferenciální rovnici pro start rakety v zadaném prostředí

$$m(t)\ddot{h}(t) = F - m(t)g \quad (2\text{-}7)$$

2.3.1 Hmotnost rakety v čase

Hmotnost v čase můžeme určit pomocí počáteční hmotnosti rakety a rychlosti spalování.

$$m(t) = m_i - rt$$

Tímto omezíme časový interval, pro který můžeme pohybovou rovnici v souladu se zadáním použít[4], na $t \in [0, t_f]$, kde t_f vyjádříme pomocí finální hmotnosti m_f.

[3] Zrychlení ve směru $\vec{e}_h$

[4] Pevně zadaná finální hmotnost omezí časový interval.

$$m(t_f) = m_f = m_i - rt_f$$

$$t_f = \frac{m_i - m_f}{r} \tag{2-8}$$

2.3.2 Dosazení hmotnosti do pohybové rovnice

Pohybovou rovnici upravíme pro získanou hmotnost a osamostatníme $\ddot{h}(t)$. Nesmíme zapomenout, že palivo spalujeme do vyčerpání paliva v t_f, získaná pohybová rovnice tedy platí pro $t \in [0, t_f]$.

$$\ddot{h}(t) = \frac{F}{m_i - rt} - g \tag{2-9}$$

3 Řešení problému

3.1 Analytické řešení

Vzhledem ke tvaru diferenciální rovnice můžeme řešit přímo dvojitou integrací podle času, tím dostaneme obecné řešení

$$h(t) = \frac{Fm_i}{r^2} \ln\left(m_i - rt\right) + \frac{F}{r} \ln\left(m_i - rt\right)t - \frac{gt^2}{2} + Ct + D$$

kde C a D jsou konstanty podle počátečních podmínek ze zadání $h(0) = 0\,\mathrm{m}$ a $\dot{h}(0) = 0\,\mathrm{m\,s^{-1}}$. Dosazením dostaneme po úpravě celé analytické řešení ve tvaru

$$h(t) = \frac{Fm_i}{r^2} \ln\left(1 - \frac{rt}{m_i}\right) + \frac{F}{r} \ln\left(\frac{m_i}{m_i - rt}\right)t - \frac{gt^2}{2} \tag{3-1}$$

Analytické řešení jsme také získali přímo v pythonu, metoda řešení je popsána v přiloženém kódu.

3.1.1 Finální rychlost rakety

Derivací analytického řešení můžeme určit rychlost rakety v čase

$$\dot{h}(t) = \frac{F}{r} \ln\left(\frac{m_i}{m_i - rt}\right) - gt \tag{3-2}$$

Pro finální rychlost $\dot{h}(t_f)$ za předpokladu $g = 0\,\mathrm{N\,kg^{-1}}$ dostáváme

$$\dot{h}_f = \frac{F}{r} \ln\left(\frac{m_i}{m_f}\right) \tag{3-3}$$

kde F je podle třetího Newtonova zákona síla působící na expandující plyn při opuštění raketového motoru a r je rychlost spalování paliva podle hmotnosti.

$$\frac{F}{r} = \frac{m_e \frac{v_e}{t_e}}{\frac{m_e}{t_e}} = v_e$$

kde m_e je hmotnost paliva spotřebovaná za časový interval t_e a v_e je výtoková rychlost zplodin.

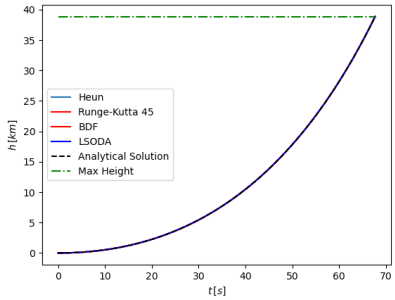
Po úpravě výrazu $\frac{F}{r}$ a zpětném dosazení dostáváme Ciolkovského rovnici 1-1 ze zadání. Potvrdili jsme vztah pro finální rychlost pro nulové gravitační zrychlení.

3.2 Numerické řešení

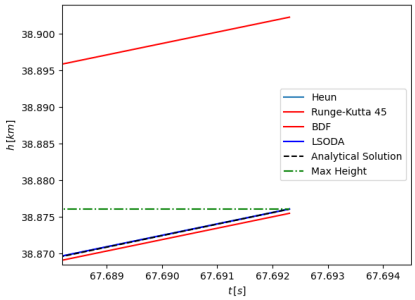
Numerické metody, které jsme pro řešení použili řeší lineární diferenciální rovnice prvního řádu a jejich soustavy. Diferenciální rovnici druhého řádu proto převedeme na soustavu diferenciálních rovnic prvního řádu, přeznačíme přitom $v(t) = \dot{h}(t)$ pro lepší přehlednost.

$$\begin{aligned} \dot{v}(t) &= \frac{f}{\mu - r \cdot t} - g \\ \dot{h}(t) &= v(t) \end{aligned} \tag{3-4}$$

Tuto soustavu jsme vyřešili pomocí numerických metod popsaných v teoretickém úvodu. Z řešení jsme vybrali pouze vzdálenost rakety od počátku, výšku, v čase. Získané řešení jsme vykreslili do grafu. Pro všechny metody jsou výsledné závislosti téměř totožné, až v přiblížení grafu najdeme odchylku od analytického řešení.



Obrázek 1: Graf řešení všech použitých metod



Obrázek 2: Přiblížený graf řešení všech použitých metod

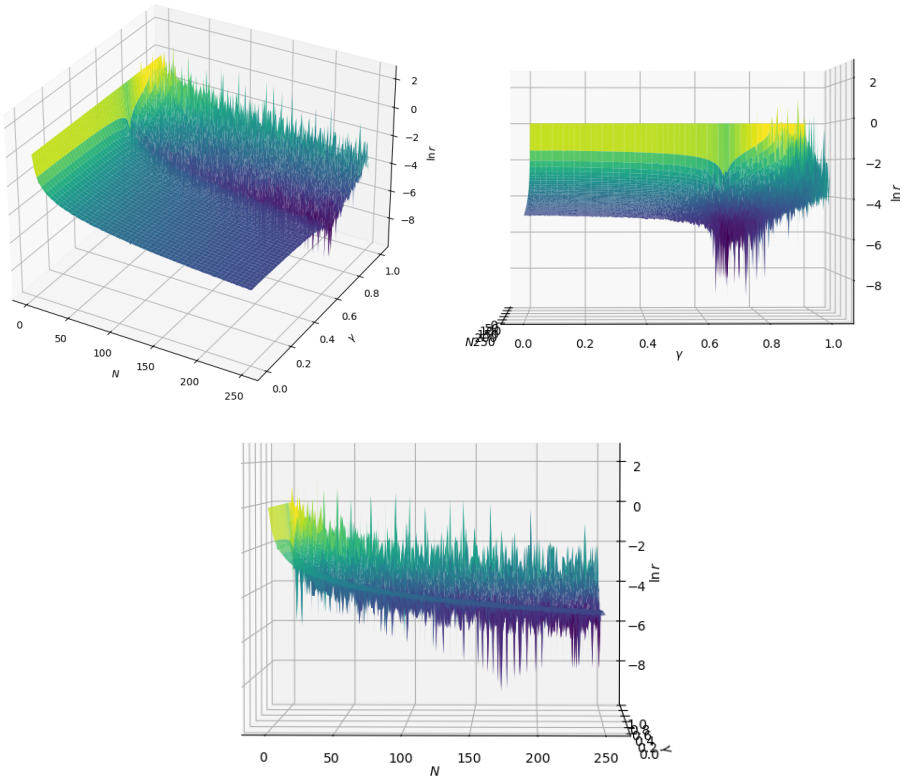
I po přiblížení tak, abychom stále viděli všechny metody, jsou Heunova metoda, LSODA a analytické řešení nerozlišitelné. Pro Heunovu metodu jsme použili parametr $\gamma = 0.67$ a $N = 100$, parametr γ jsme určili pomocí analýzy relativní chyby metody popsané později. Počet kroků jsme zvolili tak, aby heunova metoda byla přesnější než Runge-Kutta 4 z pythonovských knihoven, znovu určeno pomocí analýzy relativní chyby metod popsané později.

3.3 Určení relativní chyby Heunovy metody

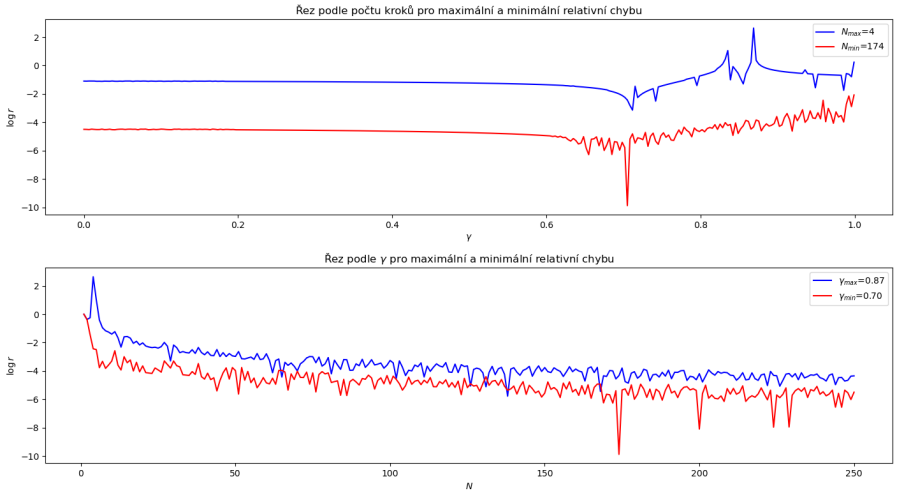
Jelikož má problém i přesné analytické řešení, můžeme relativní chybu metod určit přesně. Porovnáme finální hodnotu získanou každou metodou h_m s přesným řešením $h_f = h(t_m)$ ve tvaru

$$r = \frac{h_f - h_m}{h_f} \tag{3-5}$$

Pro Heunovu metodu jsme určili relativní chybu pro interval $N \in [1, 250]$ a 300 ekvidistantních hodnot $\gamma \in [0, 1)$. Logaritmus relativní chyby jsme vykreslili do 3D grafu, tento graf nemůžeme dát do této práce celý, proto jsme získali boční pohledy na graf a udělali řezy pro hodnoty N a γ kde nastává minimum a maximum relativní chyby. Relativní chybu v následujícím grafu uvádíme v logaritmickém měřítku o základu 10, porovnáváme tedy řády relativních chyb.



Obrázek 3: Relativní chyba Heunovy metody v závislosti na N a γ



Obrázek 4: Řez grafem relativních chyb heunovy metody

Pro takto určené parametry byla minimální hodnota relativní chyby

$$r = 1,31 \times 10^{-10}$$

a dosáhli jsme jí v bodě

$$N = 174$$

$$\gamma = 0.70$$

Ve složce s programem jsou staženy grafy pro jiné intervaly pro porovnání a soubor, do kterého se zapisují maxima a minima při každém spuštění výpočtu, vzhledem k jeho náročnosti a výpočetní době. Program také automaticky uloží mapu relativních chyb do .csv souboru.

Vzhledem k zaokrouhlování a velmi malých čísel relativní nejistoty mohou být velikosti hodnot trochu jiné i při opakovaném spuštění pro stejné počáteční podmínky. Z nějakého důvodu také pro všechny metody a to i pro metody z pythonovských knihoven nastává maximum chyby pro 4 kroky, nebyly jsme ale schopni důvod najít, tak malý počet kroků ale nemá většinou cenu uvažovat.

Obecně můžeme říct, že chyba s rostoucím počtem kroků má klesající trend, pro hodnoty do přibližně $N = 70$ kroků ale dochází k minimu chyby pro $N = 24$, nejspíše vzhledem k vnitřnímu zaokrouhlování a charakteru problému[5]. To stejné poté nastává pro hodnoty $N \in [175, 251]$, kde nejmenší hodnota nenastává pro maximální počet kroků ale pro $N = 174$. Tato skutečnost se bude opakovat i dále, ověřili jsme ji ještě pro jeden větší rozsah. Přesné hodnoty jsou uvedeny ve složce s kódem, přesná poloha a popis je uvedena v popisu kódu. Pokud tyto hodnoty minim neznáme stále platí, že optimální hodnotu dostaneme pro větší počet kroků, i když znalostí těchto minimálních parametrů můžeme dosáhnout

[5]Pro specifické počty kroků se můžeme přiblížit k řešení více.

chyby až o dva řády menší. Z výsledného grafu můžeme určit optimální hodnotu pro libovolný počet kroků $\gamma \sim 0.67$, koeficient pro několik výpočtů se pohybuje v intervalu $\gamma \in [0.66, 0.71]$. V grafu pro chybu dochází okolo této hodnoty k výraznému poklesu relativní chyby až o několik řádů.

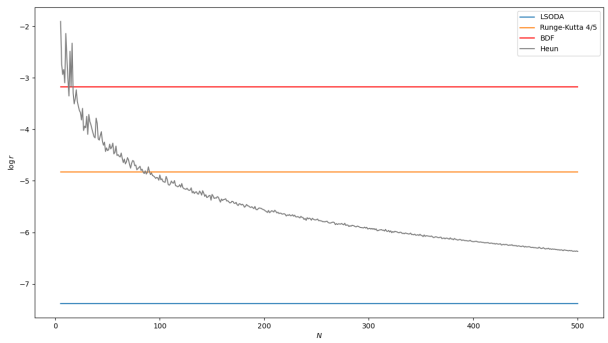
Tuto metodu analýzy chyby, který je charakteristický pro každou soustavu, si můžeme dovolit pouze protože existuje analytické měření, se kterým můžeme chybu porovnat, jinak bychom museli k problému přistoupit trochu jinak. Museli bychom porovnávat s hodnotou numerického řešení pro velmi velký počet kroků nebo přesnější numerickou metodu jako je LSODA, pro které se řešení přiblíží reálné hodnotě. Takto bychom získali odhad relativní chyby. I touto metodou by se nám ale nakonec podařilo najít optimální hodnotu parametru γ[6].

Poznámka

Analýzu chyby jsme zopakovali i pro větší rozsah, graf pro menší rozsah byl ale reprezentativnější, rozhodli jsme se proto uvést ten. Ve složce s kódem je matice s hodnotami relativní chyby pro větší rozsah, kterou můžeme zpětně zobrazit. Uložit výslednou mapu jsme se rozhodli vzhledem k časové náročnosti výpočtu. Soubor s daty je pojmenován pomocí použitého rozsahu N a γ. V přiložených .png souborech jsou také řezy pro maximum a minimum, tři vybrané pohledy na 3D graf a hodnoty maxima a minima.

3.4 Určení relativní chyby metod a jejich porovnání

Pro všechny použité metody jsme pro počet kroků N v intervalu $N \in [5, 500]$ určili relativní chybu metody. Pro metody z pythonovských knihoven je relativní nejistota konstantní, metody mají adaptivní dělení intervalu ve smyslu, že metoda interně upraví počet kroků pro lepší přesnost a konvergenci.



Obrázek 5: Relativní chyba v závislosti na počtu kroků

[6]Tvar 3D grafu by byl stejný, ale hodnoty chyb by neodpovídali reálné chybě metody

Při určování relativní chyby Heunovy metody jsme použili $\gamma = 0.67$, tento parametr se pohybuje okolo optimální hodnoty, kterou jsme získali v rozboru relativní chyby Heunovy metody. Pro $N > 100$ také relativní přesnost Heunovy metody překročí přesnosti pythonovské metody Runge-Kutta 4, proto jsme při výpočtu řešení použili krok $N = 100$.

4 Závěr

Zadaný problém jsme vyřešili analyticky a ověřili Ciolkovského rovnici pro specifikované podmínky. Poté jsme vyřešili problém numericky pomocí metod z pythonovských knihoven a vlastní verze Heunovy numerické metody. Porovnali jsme relativní nejistoty metod a určili jsme oblast pro kterou je naše metoda srovnatelná s pythonovskými metodami, počet kroků $N > 100$ a $\gamma \sim 0.67$, porovnáním s analytickým řešením. Pro naši metodu jsme také analyzovali relativní chybu v závislosti na počtu kroků a parametru γ. Našli jsme několik bodů, pro které relativní chyba rapidně klesne vzhledem k vnitřnímu zaokrouhlení a charakteru metody, stále ale přesnost roste s rostoucím počtem kroků. Dále jsme pozorovali rapidní nárůst přesnosti okolo $\gamma \sim 0.67$.